Progress Report 1

Sergio Herreros Mario Kroll Nicolás Villagrán Daniel Sánchez

April 14, 2024

1 Introduction

For this first deadline we wanted to have all the models programmed. However, we had some problems implementing the data and transforming them to piano-roll, so the development of some models had to be postponed. We will explore the implementation of the models finished and how we will perform the evaluation of the models.

2 Music Generation Taxonomy

We begin exploring different music generation techniques to decide which one is best for our particular case.

2.1 Audio Domain vs Symbolic Domain

Audio-domain means representing music as raw audio, encoded using the discrete furrier transform. With it, we can represent all possible sounds (if choosing a sampling frequency higher than human threshold, i.e 20KHz), but also require huge amounts of data and computation.

Symbolic-domain means representing music using notes, events, or text tokens to represent melodies. These representations introduce a high inductive bias, because the model can focus directly on how melodies are created. The advantage of this representation is that it is much more computationally efficient, and smaller models can generate reasonably good music. This representation also has downsides. It introduces a constraint on the possible sounds it can generate, like vibratos or hammer ons, and it also forces us to generate choose a fixed number of instruments.

We are going to use symbolic domain representation because we want to use small models and still get good results.

2.2 Conditional vs Unconditional Generation

Another aspect to consider is how much control do we want to have over the music we generate. Conditional music generation allows us to generate music based on a chord progression, a specific music genre, a sentiment, a prompt, etc.

Unconditional music generation doesn't give us any control, and we can only generate random music that will resemble our dataset. In auto-regressive models like RNN or transformers, we can slightly condition music by giving it a starting melody, and ask it to continue it, but we cannot do this with other models like an Auto-encoder or a GAN.

Since conditional generation requires more complex models and more complex datasets we will only implement unconditional models.

3 Data and pre-processing

In the previous section we decided how we are going to approach this problem. We are going to train different models to generate symbolic music without conditioning on anything.

There are two one more aspects to decide. First, which and how many instruments to generate. We are going to generate single-instrument piano tracks.

Secondly, which type of symbolic representation to use. We decided to use a piano-roll representation, since its the most common representation used in the literature. It consists on a 2D matrix, where one dimension represents discretized time and the other one the pitch of each note from to A0 to C8, exactly 88 notes representing the 88 keys of a piano. Each value at each time step is a float between 0 and 1 representing the velocity of that note.

With this information, we chose a dataset: GiantMIDI-Piano, which contains 7000 songs of 1000 composers. Music is represented in MIDI, so we need to first preprocess the dataset to convert it to piano-roll by discretizing the time dimension with a sampling frequency of 16 Hz. We also reduced the size of the dataset picking only the top 10 composers.

We created a complete pipeline inside data.py, where we go from midi to piano-roll, from piano-roll back to midi and from midi to audio in .wav format.

4 Models

We will describe here the implementation of the models and how they have done so far.

4.1 GAN models

Generative adversarial networks (GANs, for short) are a type of model used for generating data out of random noise that is indistinguishable from other (called real) data. In our case, as it is the aim of this project, we will use this model to generate music as similar as possible (and, at the same time, original) to real music scores.

GANs work by facing two models against each other. One of them is called the generator, and the other is the discriminator. The generator's objective is to *generate* data as similar as possible as the original, while the discriminator's is to *discriminate* between real and fake data, i.e. label fake data as 0 and real data as 1. So the input of the generator is random noise (every random noise vector is associated by the generator to an output; that way, it can generate different scores), and the input of the discriminator is a music score, be it real or fake.

The equilibrium of the game is when the generator outputs music scores perfectly resembling real music, and so the discriminator cannot distinguish between real and fake data. It will then always guess with 50% confidence.

Thus, the loss function for the model must be something that takes into account both how close the discriminator's labels for the real data are to 1 and how close the labels for the fake data are to 0. A good way to do this is to use binary cross entropy (in fact, it is the standard for these models). Let's call the discriminator D, the generator G, the distribution underlying the real data p_{data} and the distribution underlying the generator's outputs p_z. Then, the loss function would look like this:

$$\min_D \max_G L(D,G) = \mathbb{E}_{x\sim p_{data}(x)}[\log D(x)] + \mathbb{E}_{z\sim p_z(z)}[\log(1 - D(G(x))]$$

The solution to this minimax game would be when $p_{data} = p_g$, and so the discriminator would always guess randomly. However, this will not always be the case, and in fact it is very easy for the discriminator to "overpower" the generator, rendering the model useless. There are many techniques used and actively researched to make both the generator better and the discriminator worse, in order to give the generator a fighting chance.

4.1.1 CNN+GAN

There are many ways the generator and discriminator can process and interpret music. One of them is by using convolutional neural networks (CNNs). The model we will be implementing in this section is a version of MidiNet (mention), in which both the discriminator and the generator are CNNs.

However, CNNs applied to music generation (or anything sequential, really) pose a clear problem. That is, that they have a limited (and fixed) context size. And although this is also the case for attention based models, attention relates every data point (note) in their context size with every other, while CNNs only usually relate notes close to each other. As proposed in the MidiNet paper, we will fix this by conditioning the generator with the previous bar.

For that, we will use another model: the conditioner. This model has as input the previous bar to the one we want to predict, and its outputs are concatenated in the different layers of the generator. So, while the generator is made of a few transposed convolutional layers, the conditioner is its mirror. The first transposed convolutional layer of the generator will be the last (regular) convolutional layer of the conditioner, the second of the generator will be

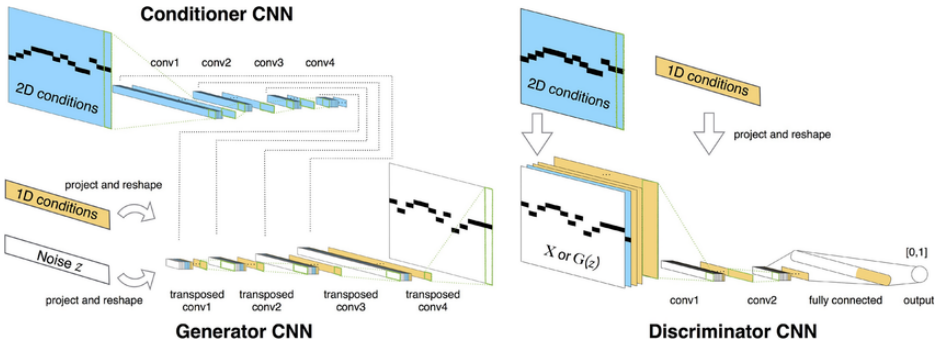

Figure 1: MidiNet

the second to last of the conditioner, etc. And the output of each of these layers is concatenated to the respective generator's layer.

The discriminator is also a normal CNN with some convolutional layers and some fully connected layers. But in order not to allow it to overpower the generator, it only has two convolutional layers, and one fully connected layer. The generator has two fully connected layers and four transposed convolutional layers, and the conditioner has four convolutional layers. After every layer, we perform batch normalization, and the regularization is leaky relu for the discriminator and the conditioner and normal relu for the generator.

We will be using other techniques for aiding the training of the GAN. The first one is feature matching. This is a form of "inverse" regularization, where the higher the lambda, the more similar the output will be to real data (albeit with loss of creativity). This is done by adding the term:

$$\lambda \|\mathbb{E}(x) - \mathbb{E}(G(z))\|_2^2$$

to the loss of the generator. The second is one-sided label smoothing, i.e. instead of setting the label of the real data as one, we set it to a slightly lower number, such as 0.9. The last technique is to, in every training iteration, train the discriminator once and the generator and conditioner twice.

4.2 Transformer

We wanted to implement a transformer model for this project. However, we found ourselves with a problem. When inferring, transformers receive input data in the encoder and the decoder tries to map it to the task its been assigned. The input data of the encoder stays the same until the decoder generates the end token. The input data of the decoder, however, changes while it generates tokens. The decoder can always *look back* to the information of the encoder. This works well when in most of the cases, because the inferring of the decoder is conditioned by the encoder.

In our case, we want to generate music from scratch so we realized it did not make sense to implement an encoder, since the decoder uses the encoder as a *reference*. Therefore, we will dispense with the encoder and implement only the decoder. The decoder structure will suffer a modification, but this will be discussed ahead.

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}} + M\right)V \tag{1}$$

Equation 1: Masked Multi-Head Attention

For now, we focused on implementing the masked multi-head attention layer of the decoder. The implementation is almost identical as the multi-head attention, but a mask is added to the product before the *softmax* function. This mask depends whether we are training or inferring. If we are training the model, we prevent the encoder to *look* ahead of the song so that it can learn to predict the correct sequence. But when we are inferring, the mask becomes trickier. In natural language, padding tokens are added to the sentences to complete the missing dimensions so that matrix multiplication can be performed. In our case, the first input the decoder receives is a matrix consisting only of padding vectors. Therefore, we want the model to ignore those padding vectors as it generates the sequence, so the mask will change as the decoder infers music, letting the model focus on non-padding vectors.

$$\begin{bmatrix} 0 & -\infty & \dots & -\infty \\ 0 & 0 & \vdots & -\infty \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & 0 \end{bmatrix}$$

Lookahead mask

$$\begin{bmatrix} -\infty & -\infty & \dots & -\infty \\ -\infty & -\infty & \vdots & -\infty \\ \vdots & \vdots & \ddots & \vdots \\ -\infty & -\infty & \dots & -\infty \end{bmatrix}$$

Padding mask at iteration 1

$$\begin{bmatrix} 0 & -\infty & \dots & -\infty \\ 0 & -\infty & \vdots & -\infty \\ \vdots & \vdots & \ddots & \vdots \\ 0 & -\infty & \dots & -\infty \end{bmatrix}$$

Padding mask at iteration 2

$$\begin{bmatrix} 0 & 0 & \dots & 0 \\ 0 & 0 & \vdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & 0 \end{bmatrix}$$

Padding mask at iteration T

5 Evaluation

It can be seen that the models described before differ in structure and how they generate the music. With GANs generating music from noise to transformer generating from a start token, the performance metrics used to evaluate the models are also different from one another. Especially with GAN models, as they use a very specific loss, used only with these models. Therefore, evaluating their performance using traditional objective metrics would be a challenge.

Therefore, we decided to perform a Turing Test with the generated music from the models. The Turing Test is an evaluation of a machine that measures its similarity to a human. This is achieved by whether a human can distinguish between the computer and a real human (or a human-generated sample, in this case). In our case, we will request the human to differentiate between music generated by humans or music generated by computers.

We will first perform the test with people that do not have music knowledge. If the results turn out to be extraordinary (no virtual distinction made by the test subjects between real and fake music), we will reach out to people that indeed have some music knowledge. This way the test is more strict and the results will be much more realistic to evaluate if the models are good or not. Should we get good results even with this restriction, then the project would be a total success.